

CONTENT STUDIO

Technical Documentation & System Manual

Project Status: Active Development

Document Version: 1.0
Last Updated: May 11, 2026
Repository: github.com/AndileP3/Content-Studio

1. Project Overview

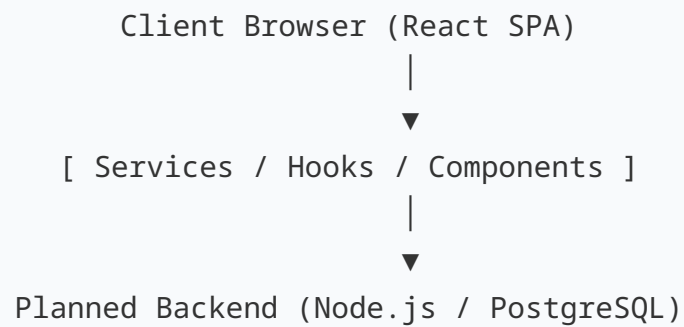
Content Studio is a modern web application designed for content creators and digital teams to manage workflows with speed and simplicity. It provides a centralized hub for content planning, creation, and media management.

1.4 Roadmap & Objectives

Objective	Priority	Target Timeline
Fast, Responsive UI	High	Q2 2026
Content Calendar	High	Q3 2026
Media Library	Medium	Q3 2026
Collaboration Tools	Medium	Q4 2026

2. System Architecture

The application follows a modern Single-Page Application (SPA) architecture, transitioning toward a full-stack environment in Phase 2.



Tech Stack (Frontend)

- **Library:** React 18.2.0
- **Language:** JavaScript (ES6+)
- **Build Tool:** Create React App 5.0.0
- **Styling:** CSS3 (BEM Convention)

3. Installation & Setup

Quick Start

```
# Clone and enter directory
git clone https://github.com/AndileP3/Content-Studio.git
cd Content-Studio

# Install dependencies
npm install

# Start development server
npm start
```

The server will run at `http://localhost:3000` by default.

4. Project Structure

A modular directory structure designed for scalability and maintainability.

```
src/
├── components/  # UI Library (Common, Layout, Forms)
├── pages/       # View-level components (Home, Dashboard)
├── services/    # API logic and external integrations
├── hooks/       # Custom React hooks
├── context/     # State management (Context API)
├── styles/      # Global CSS and Variables
└── utils/       # Helper functions and validators
```

5. Development Guide

Coding Standards

- **Components:** Use Functional Components and Hooks. PascalCase filenames.
- **CSS:** Use CSS Variables for theme consistency and BEM for naming.
- **Commits:** Use conventional commits: `feat:`, `fix:`, `docs:`.

State Management Approach

The project utilizes the **React Context API** for global state (Auth, Content) and `useState` for local component logic. This avoids heavy external libraries while maintaining clean data flow.

6. Testing Strategy

We target **80% overall coverage** using Jest and React Testing Library.

Area	Target Coverage
Utilities	90%
Hooks	85%
Components	80%

7. Build & Deployment

The production build is optimized with Terser minification and code splitting.

```
# Generate production-ready assets  
npm run build
```

Deployment Targets: Netlify (primary), Vercel, or GitHub Pages.

8. API Reference (Planned)

Future integration will support the following RESTful patterns:

Method	Endpoint	Description
POST	/api/auth/login	User Authentication
GET	/api/content	List all content items
POST	/api/content	Create new content
PUT	/api/content/:id	Update existing entry